Phase 2 Progress Report

Praxeological policy analysis: the analyser

Thin slice, rule base, LLM stages, controls, mathematical annex

1. Summary
2. Build order (plan §5.5)
3. What the analyser does now
4. LLM stages, run with subagents
5. Coverage of the worked examples
6. The mathematical annex
7. Limits
8. What needs people

Status of the Phase 2 build (plan §5) as of this report. Everything described here is in the repository: `praxeo/` (the analyser), `cases/` (inputs and runs), `tests/`, and `math-augmented-praxeo/` (the separate mathematical annex). All runs below are offline: the LLM stages were played by subagents standing in for the API (see §4).

1. Summary

- **The pipeline runs end to end.** Policy text goes through parser, policy record, context filter, typed questions, decider, rule engine (one run per reading of each unclear provision), CR-12 combiner, verifier, explainer, a frozen and hashed prediction log, and finally scoring against an outcome card.
- **Coverage of the handbook's worked examples.** From their policy records, the engine reproduces **28 of 31** consequence rows of WE-001 (a rent freeze) and **31 of 32** of WE-013 (a steel tariff). Every row not produced is listed in the tests with its reason.
- **Two configurations agree.** Configuration A (the clingo rule engine) and Configuration B (a loop of typed questions with no engine) give the same reported consequences on the San Francisco case, with group-sign and window rows excluded from the comparison.
- **LLM stages behave well after one question-schema review.** With Haiku as parser and decider, the decider's average gap from the hand-written probabilities fell from 0.21 to 0.07 (San Francisco) and from 0.20 to 0.09 (WE-013), and the full pipeline on Haiku's own record matched the baseline. On 21 control questions it answered every one on the right side of 0.5, with no bias toward "yes".
- **Nothing here is evidence of accuracy yet.** The only historical case (San Francisco 1994) is development data. Its outcome card is unverified, and the scores test the plumbing. The limits are in §7.

2. Build order (plan §5.5)

Step	Status
1. Thin slice: one rule, one worked example, one historical case, end to end with scoring	Done (San Francisco 1994; WE-001 as a consistency check)
2. A few rules both ways: engine rules and typed-question schemas, through the LLM decider	Done for price ceilings (Configuration A and B); B covers no taxes, tariffs, windows or groups yet
3. Full structured policy schema; examples run from raw text	Schema covers ceilings, per-unit taxes and tariffs. The parser runs from raw text through the LLM path; both examples parsed by Haiku

Step	Status
4. Grow the catalog one family at a time, with exceptions, questions and tests	Price controls and taxes/tariffs: 37 patterns and core rules from the 1.4 catalog (PC-1–PC-7, PC-16, PC-21, RS-1–RS-8, RS-10, RS-11, RS-14, RS-15, TX-1, TX-2, TX-4, TX-7–TX-9, OC-1–OC-4, SB-4, TS-1, TS-2, CR-4, CR-8, CR-9). Subsidies added (SB-1 to SB-6, PC-20), built on WE-019 and run out of sample on five more examples (§5). Licensing and quotas added (EN-1 to EN-3, EN-5, EN-6, EN-11), built on WE-003 with the quota rules from the catalog text, and run out of sample on five examples (§5). Prohibitions added (EN-7, EN-8, EN-10, built on WE-017). Mandates added (PC-17 built on WE-029; PC-18 and EN-14 from the catalog text) and run out of sample on five examples (§5). Play or pay, required inputs, disclosure (EN-15) and RS-24 added from the mandate runs. Price floors added (PC-9 to PC-11 as CR-6, built on WE-009). Other families (private rules) not yet encoded
5. Context grounding, combiner, verifier, full explainer	Context: date filter, retrieval per question and enrichment from sources (no real sources yet). Combiner: declared table of CR-12 comparisons. Verifier: in place. Explainer: templated sentences
4b. Core rules as the engine's main path (a change to plan 4.4, agreed during Phase 2)	Done as a first version (`praxeo/core`, `PILOT_CORE_RULES.md`): the core rules as engine rules, a situation record in CR-1's terms, an expand-as-you-go loop (LLM proposer, decider, engine), and the pipeline's default path. The patterns are kept as checks: the core rules re-derive 32 of them on small situations (`tests/test_core_derives_patterns.py`). On blind records of rent control, price caps, taxes and gouging laws, the core path equals or beats the pattern engine in every case run. On one blind record per remaining family (step 6) it beats it on quotas (18 against 16 of 54), comes close on subsidies and bans (13 against 15, 10 against 11), and falls well short on licensing (8 against 16) and mandates (13 against 29), where firm-size thresholds, play-or-pay and per-seller fees still lack a general form (table in `PILOT_CORE_RULES.md`). The LLM proposer and seed parser are written but untested against the API (no key in this environment)
6. Jev decider, calibrated on the question bank	Not started: no Jev access, no labelled question bank. In the core path Jev would answer the premise questions; proposing stays with an LLM
7. Rule proposer behind a review gate	Not started

Exit test (both configurations reproduce every Phase 1 worked example from raw text): not met. Three of 67 examples were used to build rules and thirteen more were run out of sample; Configuration B lags A, and the expert review of derivations has not happened.

3. What the analyser does now

Stage	Module	State
Parser	`parser.py`	LLM parser with structured output into the policy record. Quoted source sentences are checked in code. The prompt lists the role fields each instrument needs and a controlled list of 104 actor-class names
Vocabulary	`vocabulary.py`	Variables, instruments and actor classes are read from the handbook's ontology, the single source; rules, records and outcome cards are checked against it
Context	`context.py`, `parser.py`	Only sources dated before enactment are used. Each question gets the few most relevant to it (word-overlap retrieval). An enrichment step proposes related markets from them, every quote checked in code. No case has real sources yet
Typed questions	`questions.py`	One concept per question (rule enforced by a test). 22 questions per rent ceiling; per-instrument sets for taxes and tariffs
Decider	`decider.py`, `escalation.py`	Fixture, LLM and human deciders behind one interface. Answers below a confidence threshold escalate to a fallback and are logged

Stage	Module	State
Engine (Configuration A)	`engine.py`, `rules/*.lp`	clingo forward-chaining with provenance. Each conclusion carries its minimal assumption sets (ATMS-style), found by subset-minimal enumeration; this is exact because rules negate only facts, never assumptions, and is checked against brute force. Supports per-reading signs, group signs (at least one of several variables), announcement-window timing, and items within a variable
Combiner	`combiner.py`	A declared table of CR-12 comparisons. Each total is `ambiguous` unless its dominance premise has p ≥ 0.5
Verifier	`verifier.py`	Vocabulary, premises, cut-off, readings, groups
Configuration B	`config_b.py`	The ceiling rules as a loop of typed questions. Results are Tier J (a decision trail, not a proof)
Scoring	`scoring.py`	Headline and logic precision, recall and F1; Brier score on assumptions. Rules for ambiguous, unverifiable and group predictions. Outcome cards match actors by role
Run log	`pipeline.py`	The prediction is written and hashed before the outcome card is opened

4. LLM stages, run with subagents

No API key is available, so Haiku subagents played the parser and decider. They worked only from the policy text and the stage instructions (`tools/prep_llm_stage_inputs.py`, `tools/compare_llm_run.py`). The SDK code path itself is still unexercised.

	Round 1	Round 2 (after the question-schema review)
San Francisco decider: average gap from fixtures	0.21	0.07
San Francisco decider: answers on the other side of 0.5	3	0
San Francisco: consequences, Haiku record and answers	0 of 9	9 of 9
WE-013 decider: average gap	0.20	0.09
WE-013 decider: answers on the other side of 0.5	2	0
WE-013: consequences, Haiku record and answers	5 of 11	10 of 11

What the first round exposed:

- **Double-barrelled questions.** The enforcement question asked about a complaints body *and* penalties larger than the gain; Haiku found no penalties and answered no, which collapsed every San Francisco chain.
- **A misreadable mechanism.** Haiku read the TX-8 exchange-rate channel as an "offset" to it.
- **An engine crash.** The parser wrote prose into the field for engine facts.
- **Empty role fields.** The parser had no list of the role fields each instrument needs, so it left the tariff roles blank.

All four were fixed, and each fix has a test.

Bias check. The round-two prompt adds that a silent record is not evidence against. To check it did not bias the decider toward "yes", I wrote 15 fictional control cases with 21 labelled questions, 10 of them with correct answer "no". They come in two kinds: the deciding fact stated outright, or only implied (a cap far above a collapsing market; enforcement given to a department with no budget). Both prompts got all 21 right. Mean p was 0.08–0.09 on the "no" questions and 0.84 on the "yes" ones. These controls test reading, not calibration on genuinely open questions.

5. Coverage of the worked examples

Example	Rows	Produced	Not produced, and why
WE-001 rent freeze	31	28	Two rows rest on facts specific to this case that the record lacks (a homebuyers' market relation; the K3 relocation payment). One needs the new-construction exemption as a fact, and one of its chains rests on the negation of an assumption, which the engine keeps out to stay exact
WE-013 steel tariff	32	31	One row needs a class the record does not name. The return-on-capital row is left ambiguous by the handbook itself

Out of sample. Thirty-three more examples were encoded after their family's rules were built, without changing any rule first; subagents wrote the records and answers from the policy text and the assumption tables. Each first run is kept in the case's `out_of_sample.md`, and rules were then changed only where a gap was general (a pattern-level fix, citing the catalog).

Family (built on)	Examples run out of sample	Rows produced, first run	After general fixes	WRONG engine rows, first run	After
Price ceilings (WE-001, San Francisco)	WE-005, WE-006, WE-067	41 of 123	57 of 123	6	1
Taxes and tariffs (WE-013)	WE-002, WE-010, WE-042, WE-046, WE-062	46 of 255	60 of 255	5	3 (all the TX-8 question, Q25)
Licensing and quotas (WE-003; quotas from the catalog text)	WE-014, WE-015, WE-016, WE-028, WE-055	45 of 195	49 of 195	7	0
EN-12 and EN-13 (WE-028)	WE-039, WE-047, WE-054, WE-064	44 of 210	50 of 210	10	2 (the TX-8 question, Q25)
Prohibitions (WE-017)	WE-018, WE-037, WE-044, WE-050, WE-053	45 of 221	46 of 221	28 (keys)	11
Subsidies (WE-019)	WE-020, WE-021, WE-033, WE-056, WE-065	42 of 191	63 of 191	16	3
Mandates (WE-029; PC-18 and EN-14 from the catalog text)	WE-031, WE-034, WE-036, WE-059, WE-061	53 of 236	67 of 236	23 (keys)	5
Price floors (WE-009)	WE-008; the floor components of WE-018 and WE-061	31 of 93	37 of 93	5	0 (one more masked by a key)

Most rows not produced need roles the record cannot yet hold (related markets, factors, neighbours, foreign sellers), rules for whole families not encoded, or engine facts per reading of an unclear provision. Coverage on unseen examples is therefore low, and the first runs' WRONG rows show what a family's rules get wrong before they are corrected.

San Francisco 1994, the historical case, reports 27 consequences. Five are verifiable against its card: headline F1 1.00, logic F1 0.57, assumption Brier score 0.06.

6. The mathematical annex

`math-augmented-praxeo/` sizes consequences the praxeological rules have already signed, and never sets a sign. It rests on seven math core rules (MR-1 to MR-7: ranges, identities, compounding, first-order response, clearing as a tendency, no-arbitrage, composition of magnitudes). The admission test

is: a method belongs in the annex if and only if it can be derived from those rules with sourced input ranges.

- **Methods surveyed:** 25 adopted, 4 bounds, the rest excluded (welfare in money, multipliers, equilibrium and game models).
- **Handlers:** 27, each with a derivation line and a diagnosis for when its result contradicts a sign.
- **The guard on CR-12:** only arithmetic on the case's own figures may settle a comparison. First-order results are only evidence for a premise, and a decider must accept that evidence.
- **Results:** on San Francisco and WE-013, no annex result contradicts a sign. All annex ranges are illustrative until they are sourced.

7. Limits

1. **No real test of accuracy.** San Francisco is development data (PC-21 was built from it). Its outcome card was written from memory of Diamond, McQuade and Qian (2019) and is unverified, and the policy text is a summary rather than the ordinance.
2. **Fixtures written with hindsight.** The hand-written fixture probabilities and the choice of rules were made knowing the San Francisco outcome.
3. **The LLM code path has not run.** Subagents stood in for it.
4. **Calibration is untested.** There is no labelled question bank, so confidence thresholds are placeholders.
5. **Premises added from the examples.** The engine encodes some premises that the examples' derivations need but no pattern states (`relet_obstructs`, `exit_process`, and three illustrative WE-013 probabilities). They await expert review.
6. **Narrow coverage.** Five intervention families are encoded; on unseen examples the engine produces about a quarter to a third of the handbook's rows.

8. What needs people

- **An API key:** run the LLM parser and decider through the real code path.
- **Sources:** the Diamond, McQuade and Qian (2019) paper and the Proposition I ordinance text, to verify the San Francisco card; pre-1994 data for the annex ranges.
- **A held-out historical case** with a sourced outcome card. This is the first real test of accuracy.
- **Experts:** review the handbook's open questions (Q1–Q25; Q25 is the TX-8 inconsistency between WE-013 and WE-042), the rules, and the premises in §7; label the question bank (about 200 items per question type) so thresholds can be set.